Anti-Corruption Layer

translate at the border, keep the domain clean

Watch the 4-min video

In one line: an anti-corruption layer (ACL) is a translation layer at one boundary that converts a legacy or external system's names, codes and structures into your own domain model and back, so the foreign model never leaks into your code.

1 KUNNR and status code 7

A new fulfilment service needs orders from a twenty-year-old ERP. The ERP calls a customer `KUNNR`, uses status `7` for shipped, stores weights in grams and dates as strings. If you use those models directly, every developer must learn what code 7 means, the ERP's quirks spread through your code, and your clean new domain slowly becomes the legacy system.

2 What an ACL is

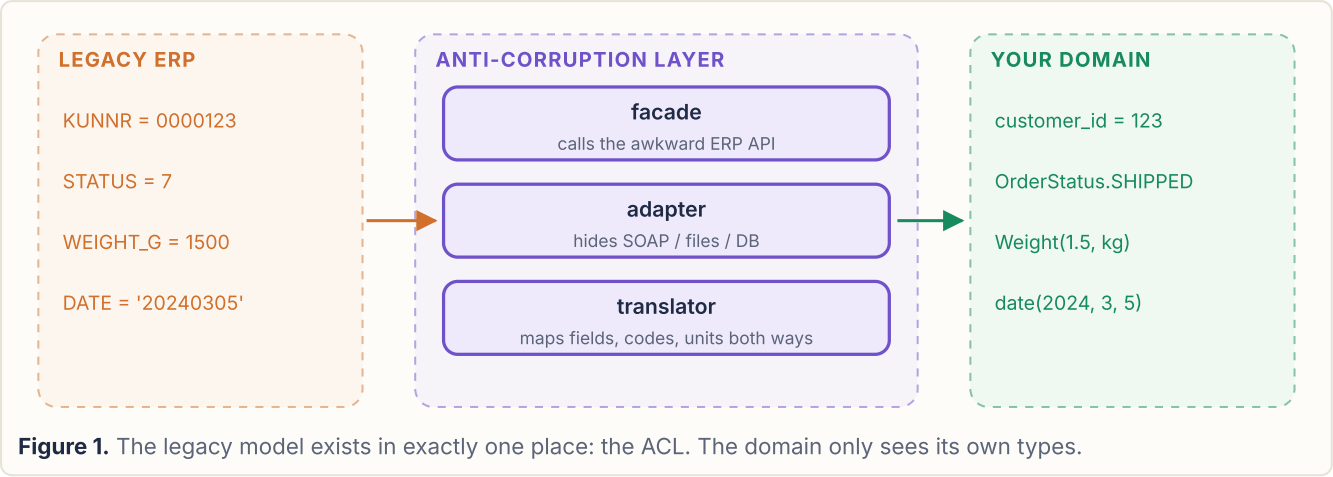


Figure 1. The legacy model exists in exactly one place: the ACL. The domain only sees its own types.

The ACL is a pattern from domain-driven design's context map (lesson 22). It sits between two bounded contexts whose models disagree and translates in both directions. Typically it has three parts:

- **Facade:** a simple interface over the legacy system's awkward API.
- **Adapters:** hide the transport: SOAP, nightly files, message formats, even a shared database.
- **Translators:** map fields, codes and units into domain objects, and back for writes.

It can be a module inside your service (often the adapter behind a port, lesson 31) or a small service of its own when several services need the same translation.

3 Why models leak, and what the ACL buys

Without an ACL	With an ACL
status code 7 checked in twenty files	`OrderStatus.SHIPPED` everywhere; code 7 in one mapping
ERP adds code 9: hunt for every switch statement	update one translator, add one test
ERP replaced: rewrite your service	replace the adapter; the domain is unchanged
legacy naming in new APIs and databases	your ubiquitous language (lesson 21) stays intact

The ACL pairs naturally with the **strangler fig** (lesson 29): new services talk to the monolith only through an ACL, and when the monolith is finally gone you delete the ACL and the domain is already clean.

COSTS AND WHEN TO SKIP IT

- **Extra code** to write, test and maintain.
- **Latency:** translation is cheap, but an ACL that makes one legacy call per field is slow. Batch calls and cache reference data.
- **Hidden data loss:** a legacy field with no clean equivalent can be silently dropped. Map it explicitly or fail loudly.
- **Skip it** when the other model already matches yours, or the integration is tiny and short-lived.

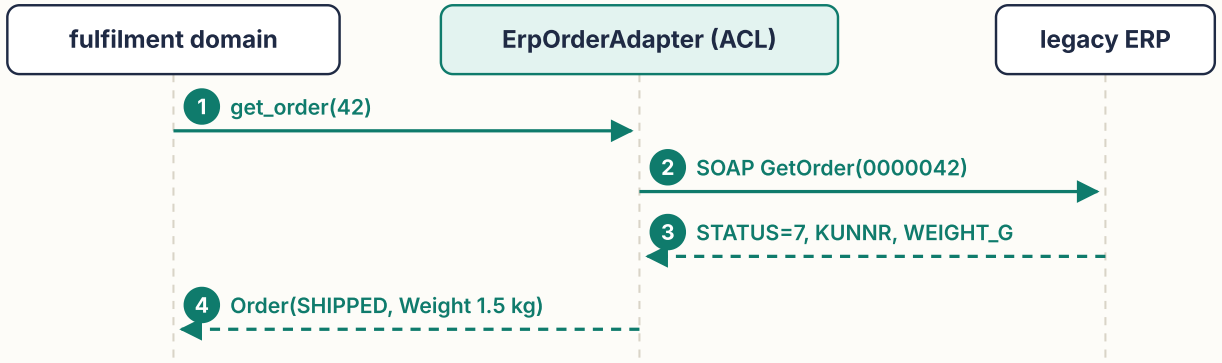


Figure 2. One read: the domain asks in its own terms; only the adapter speaks the ERP's language.

WORKED EXAMPLE: FULFILMENT READS ORDERS FROM THE ERP

An `ErpOrderAdapter` calls the ERP and returns domain objects. Its translator maps `STATUS = 7` → `OrderStatus.SHIPPED`, `KUNNR` → `customer_id` (stripping leading zeros), `WEIGHT_G = 1500` → `Weight(1.5, kg)`, and `'20240305'` → a real date. An unknown status code raises an error and an alert instead of guessing. The fulfilment domain never sees an ERP code; when the company replaces the ERP, only the adapter is rewritten.

4 Key terms

Anti-corruption layer: translator that keeps a foreign model out

Facade: simple interface over a complex system

Model leakage: foreign concepts spreading through your code

Translator / mapper: converts fields, codes and units

Adapter: hides transport such as SOAP or files

Upstream / downstream: who provides vs who consumes a model

5 Interview questions

Q1. What problem does an anti-corruption layer solve? BASIC

It stops a legacy or external model from leaking into your domain by translating names, codes and structures at one boundary.

Q2. What are the usual parts of an ACL? BASIC

A facade over the foreign API, adapters for the transport, and translators that map fields, codes and units in both directions.

Q3. Your new service must read orders from an ERP with cryptic codes. How? SCENARIO

Build an adapter with a translator that maps ERP codes and fields into domain types, fails loudly on unknown values, and is the only code that knows the ERP model.

Q4. How does an ACL help a strangler fig migration? SCENARIO

New services talk to the monolith only through the ACL, so their domain stays clean; when the monolith is gone, you delete the ACL.

Q5. When would you skip an ACL? TRICKY

When the other system's model already matches yours, or the integration is tiny; the extra code and latency then buy nothing.

Q6. Your ACL is slow. Why, and how do you fix it? TRICKY

Usually it makes many fine-grained legacy calls; batch requests, cache reference data such as code tables, and fetch only what the use case needs.

Next: SD-L-013 · Hexagonal Architecture backendroadmap.com